Merging BERT Experts with Weight-Ensembling Mixture of Experts

Few-Shot Router Training across Five GLUE Tasks

Research Report

AUTHORS (GROUP 1):

NGUYỄN BÁ ANH 20261218M

NGUYỄN THỊ NHÃ LINH 20261262M

INSTRUCTOR:

NGUYỄN KIÊM HIẾU

Hanoi University of Science and Technology

CONTENTS

Abstract **2**

Chapter 1. Introduction **3**

Chapter 2. Background and Related Work **5**
2.1 Transformer Architecture 5
2.2 BERT 5
2.3 Mixture of Experts 6
2.4 Weight Ensembling 7
2.5 GLUE Benchmark 7

Chapter 3. Proposed Methodology **8**
3.1 Overall Architecture 8
3.2 Task-specific BERT Experts 8
3.3 Router Architecture 10
3.4 Weight-Ensembling Mechanism 10
3.5 Training and Inference Pipeline 11

Chapter 4. Experimental Setup **13**
4.1 Datasets and Tasks 13
4.2 Implementation Environment 13
4.3 Experimental Configuration 14
4.4 Evaluation Metrics 14

Chapter 5. Results and Discussion **15**
5.1 Performance Evaluation 15
5.2 Comparison with Individual Experts 16
5.3 Model Storage and Efficiency 16
5.4 Analysis and Discussion 17
5.5 Limitations 18

Chapter 6. Conclusion and Future Work **19**

References **20**

Abstract

Fine-tuning one pre-trained language model per task gives strong task-specific performance, but it multiplies storage and deployment effort, while static weight merging discards much of the knowledge gained during fine-tuning. This report studies Weight-Ensembling Mixture of Experts (WEMoE) as a model consolidation method for natural language processing. Five BERT-base experts are fine-tuned independently on the MRPC, RTE, CoLA, SST-2 and QNLI tasks of the GLUE benchmark. In the merged model, non-MLP parameters are combined statically through Task Arithmetic, whereas the MLP parameters of every encoder layer are reconstructed for each input as the pre-trained weights plus a router-weighted sum of frozen task vectors. Predictions therefore come from a single forward pass with parameter-level combination and a task-specific head, not from averaged logits. Unlike the original image-based WEMoE study, which adapts routers on unlabelled test data, this implementation trains only the routers with Few-Shot Fine-Tuning on labelled examples that the experts never saw. All model selection uses a dedicated development split, and final scores are computed on the official validation splits. With 256 examples per task, WEMoE reaches an average score of 69.9 ± 0.1, against 57.4 for Task Arithmetic, 78.2 for the experts and 45.8 for a majority-class predictor. It recovers 59% of the gain between the majority baseline and the experts. Upscaling only the first MLP layer gives a similar average (69.1 ± 0.1) with 47.2% of the storage of five experts rather than 73.1%, but the two variants fail on different tasks: RTE for the full variant and MRPC for the intermediate-only one. Small labelled budgets are unstable across seeds. Limitations include a single set of experts, small validation sets for two tasks, and development data that exceeds the few-shot budget.

Keywords: model consolidation; Weight Ensembling; Mixture of Experts; BERT; multi-task learning; few-shot fine-tuning.

Chapter 1

Introduction

Pre-trained language models have changed how natural language processing systems are built. An encoder such as BERT [4] can be fine-tuned for a new classification problem with limited data, usually outperforming a task-specific network trained from scratch. An organisation supporting several language understanding tasks can therefore accumulate many fine-tuned copies of one architecture, each holding a complete parameter set even though most parameters stay close to the shared pre-trained values.

Multi-task learning [2] offers the classical alternative of one model trained jointly on all tasks, but joint training requires access to all data at once and repeats its cost whenever a task is added. Keeping independent models avoids this but imposes the storage and maintenance burden described above. These two positions motivate *model consolidation*: merging already fine-tuned models into one without returning to the full training data.

Simple Averaging and Task Arithmetic [7] merge parameters statically and therefore commit to one parameter vector for every input. Tang et al. [17] addressed this limitation with Weight-Ensembling Mixture of Experts (WEMoE), in which a small router observes each input and determines how task-specific parameter updates are combined in the MLP blocks of a Transformer. Their work evaluated CLIP vision models on image classification and trained the routers through unsupervised test-time adaptation.

This report documents a project that adapts WEMoE to BERT and the GLUE benchmark and trains the routers with Few-Shot Fine-Tuning (FSFT) on a small labelled set instead of test-time adaptation.

The research objective is to determine whether input-conditioned MLP weights can recover task performance within one merged BERT model, and at what storage cost relative to five experts. Three questions follow. How much of the performance lost by static merging can routers recover when trained on labelled examples that the experts never saw? How does recovery depend on the labelled budget? And how does the choice of upscaled MLP modules affect accuracy, storage and per-task behaviour?

The contributions are practical, and no novel algorithm is claimed. The project (i) adapts

the WEMoE router and weight-ensembling module to the post-norm BERT encoder with attention-mask-aware routing; (ii) applies a protocol that isolates expert training, router training, model selection and final reporting; (iii) evaluates two upscaling scopes and three labelled budgets; and (iv) analyses results against a majority-class floor, which exposes failures that an average conceals. The scope is limited to five binary GLUE tasks with one BERT-base backbone. Test-time adaptation is not implemented and is therefore out of scope.

Chapter 2 reviews the background, Chapter 3 presents the method as implemented, Chapter 4 specifies the experiments, Chapter 5 reports and discusses the results, and Chapter 6 concludes.

Chapter 2

Background and Related Work

2.1 Transformer Architecture

The Transformer [18], shown in Figure 2.1, replaces recurrence with attention. Its central operation, scaled dot-product attention, relates queries Q to keys K of dimension d_k and values V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{2.1}$$

The factor $\sqrt{d_k}$ prevents large dot products from pushing the softmax into regions with small gradients. Multi-head attention applies h learned projections in parallel:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \quad \text{head}_i = \text{Att}(QW_i^Q, KW_i^K, VW_i^V), \tag{2.2}$$

where Att is Equation (2.1) and W_i^Q, W_i^K, W_i^V, W^O are learned projections.

Attention is insensitive to token order, so order information is added to the input: fixed sinusoidal encodings in the original Transformer and learned position embeddings in BERT. Each layer also contains a position-wise feed-forward network, referred to in this report as the MLP block:

$$\text{FFN}(h) = W_2\,\sigma(W_1 h + b_1) + b_2, \tag{2.3}$$

where h is a token representation, σ a non-linearity, and (W_1, b_1), (W_2, b_2) the parameters of two linear layers. Every sub-layer is wrapped by a residual connection followed by layer normalisation [1], $h' = \text{LayerNorm}\big(h + \text{Sublayer}(h)\big)$. This post-norm arrangement, used by BERT, differs from the pre-norm layers of the CLIP vision models of Tang et al. [17], which affects where a router reads its input.

2.2 BERT

BERT [4] is an encoder-only Transformer that conditions each token representation on both its left and right context. A sequence pair is encoded as `[CLS]` A `[SEP]` B `[SEP]`, and each input

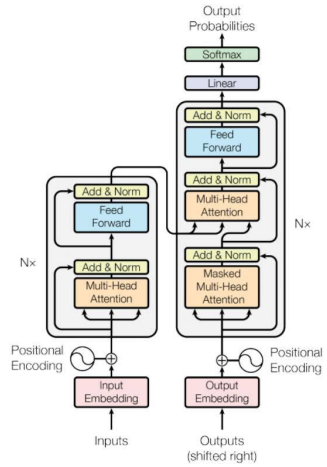


Figure 2.1: Transformer, from [18].

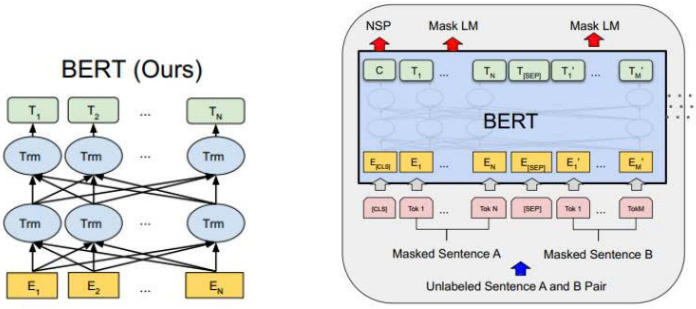


Figure 2.2: BERT: bidirectional encoder stack (left) and pre-training with Masked LM and NSP (right). Adapted from Devlin et al. [4].

position receives the sum of three learned embeddings,

$$E_i = E^{\text{token}}(x_i) + E^{\text{segment}}(s_i) + E^{\text{position}}(i), \tag{2.4}$$

where x_i is the token and $s_i \in \{A, B\}$ its segment. Pre-training combines two self-supervised objectives (Figure 2.2). Masked Language Modelling predicts the original identity of masked input tokens, and Next Sentence Prediction decides whether sentence B follows sentence A in the corpus.

The `bert-base-uncased` configuration used here has 12 layers, hidden size 768, 12 attention heads, MLP size 3072 and the GELU non-linearity [6]. For downstream classification, the pre-training heads are replaced by a new head on the pooled representation: the final `[CLS]` state passes through a dense layer with tanh activation, then dropout and a linear classifier. Because every expert in this project starts from the same pre-trained weights, the experts share one parameterisation, which makes weight-level merging meaningful.

2.3 Mixture of Experts

Mixture of Experts (MoE) models [8, 12] solve a problem through several specialised expert networks and a gating network, or router, that decides how much each expert contributes to a given input:

$$\text{MoE}(x) = \sum_{i=1}^{n} G_i(x)\, E_i(x), \tag{2.5}$$

where E_i is an expert and $G_i(x)$ its gate value. Classically, experts are trained jointly with a gate that produces a distribution, and Shazeer et al. [15] scale the idea by making the gate sparse. Equation (2.5) combines expert *outputs*; as Section 2.4 explains, WEMoE keeps the router but changes what is combined.

2.4 Weight Ensembling

Let θ_0 denote the pre-trained parameters and θ_e the parameters of expert e after task-specific fine-tuning. The task vector of expert e is its displacement from the shared starting point [7]:

$$\tau_e = \theta_e - \theta_0. \tag{2.6}$$

Task Arithmetic then merges E experts with one global scaling factor λ:

$$\theta_{\text{TA}} = \theta_0 + \lambda \sum_{e=1}^{E} \tau_e. \tag{2.7}$$

Simple Averaging is the special case $\lambda = 1/E$. These merges are cheap, but updates from different tasks may interfere and one parameter vector must serve all inputs. WEMoE [17] instead treats task vectors as a dictionary and reconstructs selected parameters for each input, restricting this to MLP blocks because fine-tuning changes them most.

Weight ensembling must be distinguished from output or logit ensembling. A logit ensemble runs every model and combines predictions, so its cost grows with the number of experts and it requires a shared output space. A weight ensemble combines parameters before they are applied, so one forward pass suffices. The two are not equivalent: for one linear layer $(\sum_e w_e W_e)h = \sum_e w_e(W_e h)$, but this equality is lost across non-linear activations and stacked layers. The price is that task vectors must remain stored and per-input weights must be assembled at run time.

2.5 GLUE Benchmark

The General Language Understanding Evaluation benchmark [19] collects language understanding tasks under common splits and metrics. This project uses five binary tasks: MRPC decides whether two sentences are paraphrases [5]; RTE decides whether a premise entails a hypothesis [3]; CoLA judges the grammatical acceptability of a sentence [20]; SST-2 classifies the sentiment of movie-review sentences and phrases [16]; and QNLI, derived from SQuAD [14], decides whether a sentence contains the answer to a question.

Chapter 3

Proposed Methodology

This chapter describes the method as implemented. Differences from the original WEMoE study are collected in Table 3.1 at the end of the chapter. Throughout, $E = 5$ denotes the number of experts, e indexes experts, t is the task of an input, and $\ell \in \{1, \ldots, 12\}$ indexes encoder layers.

3.1 Overall Architecture

Figure 3.1 shows the merged model. An input passes through one BERT encoder. Outside the MLP blocks, all parameters (embeddings, attention, layer normalisation and pooler) are static and merged by Task Arithmetic. In each encoder layer, a router reads the token states leaving the attention block and emits gating weights that determine the MLP parameters used for that input. The task identifier selects a frozen head at the end. The experts themselves are never executed at inference: they contribute only task vectors and heads when the model is built.

3.2 Task-specific BERT Experts

Each expert results from standard task-specific fine-tuning of `bert-base-uncased` (Figure 3.3). A sequence classifier with a new two-class head is trained on the expert-training portion of its task, updating all layers, and the best of three epochs is selected on the development split. The output is a pair (θ_e, g_e), where g_e is a dropout-plus-linear head producing two logits. The displacements τ_e form the dictionary of task knowledge, and the heads provide the task-specific outputs of the merged model.

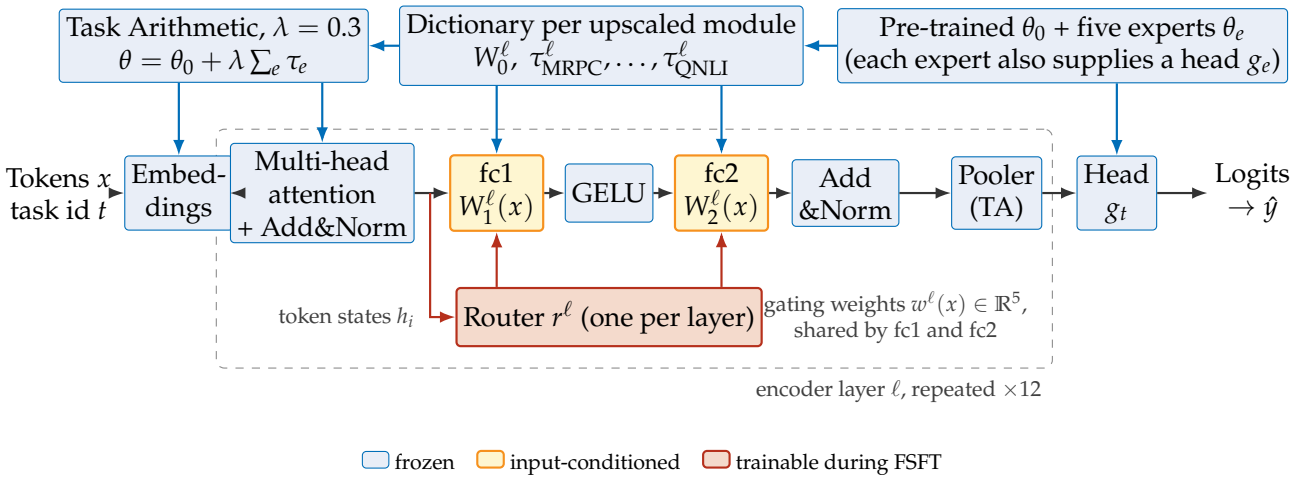


Figure 3.1: Merged WEMoE model as implemented. In the full scope both fc1 (`intermediate.dense`) and fc2 (`output.dense`) are input-conditioned. In the intermediate-only scope fc2 is static and uses Task Arithmetic. Only the routers are trainable.

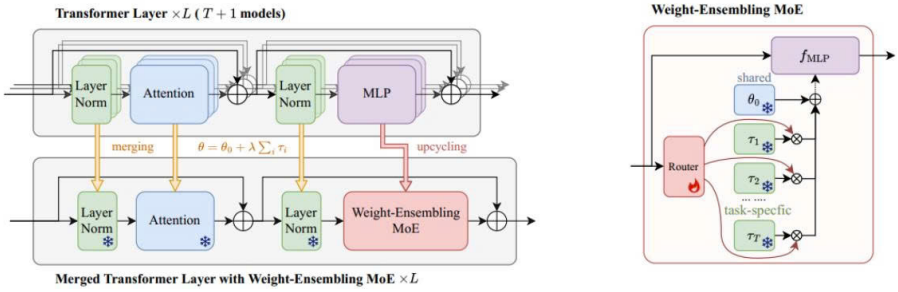


Figure 3.2: Original WEMoE design on a pre-norm Vision Transformer, reproduced from Tang et al. [17] for comparison. The implementation in Figure 3.1 applies the same principle to post-norm BERT layers.

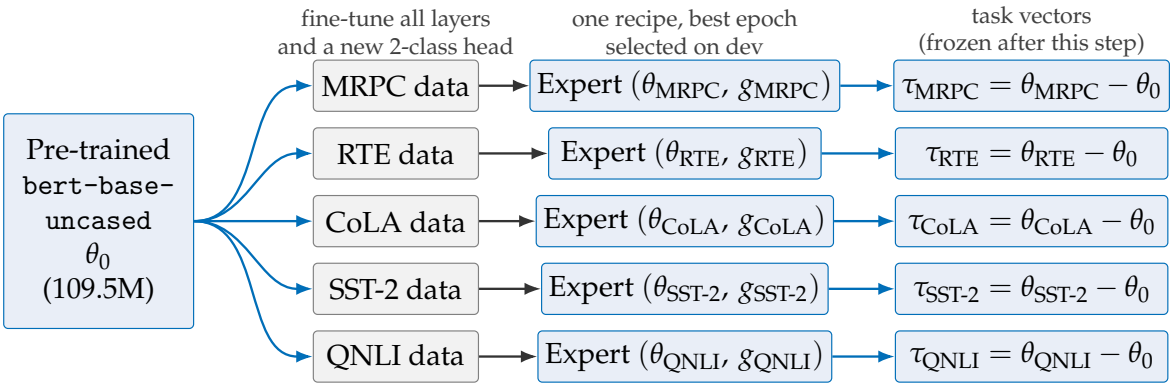


Figure 3.3: Expert preparation. Each expert is fine-tuned independently from the shared pre-trained model, then reduced to a task vector and a classification head.

3.3 Router Architecture

Each encoder layer has its own router r^ℓ, shown in Figure 3.4. Its input is the sequence of token states $h_i^\ell \in \mathbb{R}^{768}$ produced by the layer's attention block after Add&Norm. The router is a two-layer MLP applied to every token:

$$r^\ell(h) = A_2^\ell \,\mathrm{ReLU}\Big(A_1^\ell h + c_1^\ell\Big) + c_2^\ell, \tag{3.1}$$

with $A_1^\ell \in \mathbb{R}^{768\times768}$ and $A_2^\ell \in \mathbb{R}^{5\times768}$. Token outputs are averaged over non-padding positions using the attention mask $m_i \in \{0,1\}$ to obtain one vector of gating weights per input:

$$w^\ell(x) = \frac{\sum_i m_i\, r^\ell(h_i^\ell)}{\sum_i m_i} \in \mathbb{R}^5. \tag{3.2}$$

The gating weights are *unnormalised*: with no softmax and no top-k selection, a router may amplify a task vector ($w_e > 1$) or subtract it ($w_e < 0$), so the combination is neither convex nor sparse. Router weights are initialised from $\mathcal{N}(0, 0.01^2)$, with $c_1^\ell = 0$ and $c_2^\ell = \lambda$, which makes $w_e^\ell(x) \approx \lambda$, so the model begins close to Task Arithmetic. The twelve routers hold 7,133,244 trainable parameters in total.

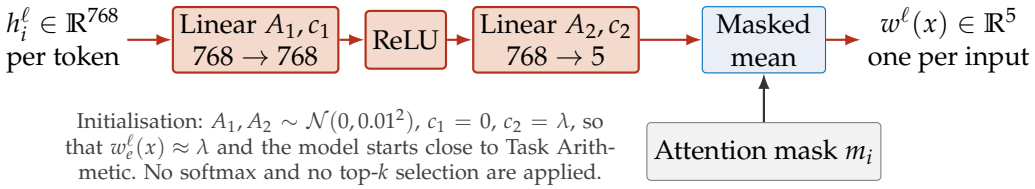


Figure 3.4: Router of layer ℓ. The MLP is applied per token and its outputs are averaged over non-padding tokens.

3.4 Weight-Ensembling Mechanism

Let $W_{m,0}^\ell$ and $b_{m,0}^\ell$ be the pre-trained weight and bias of MLP module $m \in \{1,2\}$ in layer ℓ (fc1 and fc2), and let $\tau_{e,m}^\ell$ and $\tau_{e,m}^{\ell,b}$ be the corresponding parts of the task vector of expert e. For an input x, the parameters actually applied are

$$W_m^\ell(x) = W_{m,0}^\ell + \sum_{e=1}^{E} w_e^\ell(x)\,\tau_{e,m}^\ell, \qquad b_m^\ell(x) = b_{m,0}^\ell + \sum_{e=1}^{E} w_e^\ell(x)\,\tau_{e,m}^{\ell,b}. \tag{3.3}$$

The MLP of Equation (2.3) then becomes input-conditioned, with the same gating vector shared by fc1 and fc2:

$$\mathrm{FFN}^\ell(h_i; x) = W_2^\ell(x)\,\mathrm{GELU}\Big(W_1^\ell(x)\,h_i + b_1^\ell(x)\Big) + b_2^\ell(x). \tag{3.4}$$

Parameters outside the upscaled modules, collected as $\bar{\theta}$, are fixed by Equation (2.7) at the selected λ. The intermediate-only scope conditions fc1 alone, and its fc2 also belongs to $\bar{\theta}$.

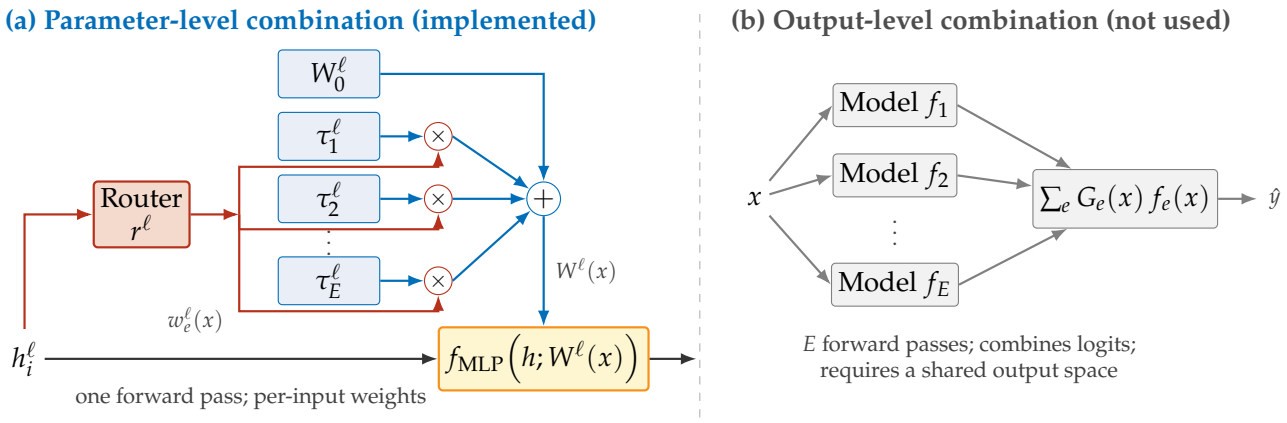


Figure 3.5: Parameter-level combination used by the implementation (a), contrasted with output-level combination (b). The latter is shown only for reference and is not part of this project.

Figure 3.5 shows the distinction that matters here. The code performs parameter-level combination only: no expert logits are computed and nothing is averaged at the output. The final prediction is

$$p(y \mid x, t) = \text{softmax}\Big(g_t\Big(\text{pool}(\text{BERT}(x; \bar{\theta}, \{W^\ell(x)\}))\Big)\Big), \qquad \hat{y} = \arg\max_y \; p(y \mid x, t), \quad (3.5)$$

where g_t is the head of the known input task. In the code, `WeightEnsembledLinear` stores W_0 and the stacked task vectors as non-trainable buffers. The default `merged` mode builds per-sample weight tensors and applies a batched matrix product; `decomposed` and `loop` modes compute the same result for memory-constrained training and for tests. Unit tests confirm that the modes agree, that constant routing reduces to Task Arithmetic and that padding does not affect routing.

3.5 Training and Inference Pipeline

Figure 3.6 summarises the workflow. Training-time operations consist of three optimisation or selection problems. First, each expert minimises the cross-entropy on its own training data $\mathcal{D}_e$:

$$\min_{\theta_e, g_e} \; -\frac{1}{|\mathcal{D}_e|} \sum_{(x,y)\in\mathcal{D}_e} \log p_e(y \mid x). \quad (3.6)$$

Next, the scaling factor of Equation (2.7) is chosen by grid search on the development split, and the WEMoE model is built from it. Finally, FSFT trains the router parameters ϕ of Equation (3.1) in all twelve layers while every other parameter remains frozen. At each step one mini-batch B_t is drawn from the k labelled examples of each task, and the summed cross-

entropy is minimised:

$$\min_{\phi} \mathcal{L}_{\text{FSFT}}(\phi) = \sum_{t=1}^{E} -\frac{1}{|B_t|} \sum_{(x,y)\in B_t} \log p_\phi(y \mid x, t). \tag{3.7}$$

Backbone dropout is disabled during this stage. The mean development score is evaluated at initialisation and every 50 steps, and the router state of the best step is restored at the end. The original paper instead minimises prediction entropy on unlabelled test data [17]; this objective is not implemented in this project.

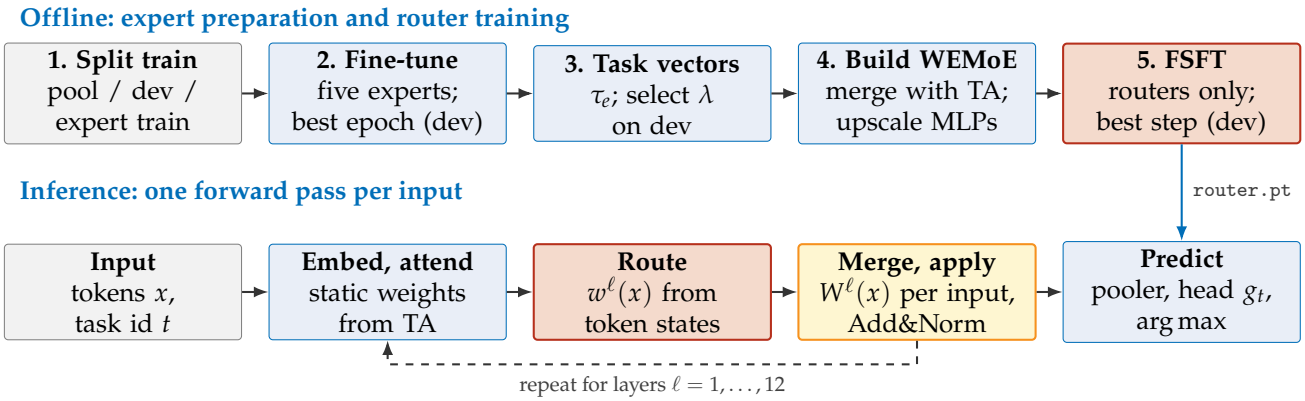


Figure 3.6: Training workflow (top) and inference workflow (bottom). Merging and training happen once; routing and weight construction happen for each input in each layer.

Inference involves no optimisation. In each layer the model attends, computes $w^\ell(x)$ from Equation (3.2), constructs the parameters of Equation (3.3) and applies Equation (3.4), and it finally evaluates Equation (3.5). A checkpoint holds only `router.pt` and a configuration file, so `load_trained_wemoe` rebuilds the model from the experts and the saved router weights.

Table 3.1: The original WEMoE study of Tang et al. [17] and this project.

Aspect	Tang et al.	This project
Backbone, tasks	CLIP vision models, image classification	BERT-base, five GLUE tasks
Upscaled modules	Transformer MLPs	Full MLP (default); intermediate-only as an ablation
Routing input	Mean over tokens	Masked mean over non-padding tokens (default)
Router training	Test-time adaptation (entropy on unlabelled test data)	FSFT (cross-entropy on labelled examples)

Chapter 4

Experimental Setup

4.1 Datasets and Tasks

All data are loaded from `nyu-mll/glue` through the Hugging Face `datasets` library [9]. Table 4.1 lists the five tasks. The official training split of each task is permuted once with a fixed seed and divided into a few-shot pool of 256 labelled examples for router training, a development split of 10% capped at 1,000 examples, and the remaining expert-training data. The experts never see the pool or the development split. The development split makes every model choice (expert epoch, Task Arithmetic λ, router step), and the official validation split is used only for reported scores. GLUE test labels are not used.

Table 4.1: GLUE tasks and data splits. Train and validation sizes and majority rates were verified against the dataset; other sizes follow from the split rule.

Task	Input	Task score	Train	Pool	Dev	Expert train	Validation	Majority (%)
MRPC	Sentence pair	mean(acc, F1)	3,668	256	367	3,045	408	68.4
RTE	Sentence pair	accuracy	2,490	256	249	1,985	277	52.7
CoLA	Sentence	MCC	8,551	256	855	7,440	1,043	69.1
SST-2	Sentence	accuracy	67,349	256	1,000	66,093	872	50.9
QNLI	Question, sentence	accuracy	104,743	256	1,000	103,487	5,463	50.5

4.2 Implementation Environment

The project is implemented in Python using PyTorch, Hugging Face Transformers [21], `datasets` and scikit-learn. The reported run used Google Colab with one NVIDIA Tesla T4 GPU (15 GB), with torch 2.11.0, transformers 5.20.2, datasets 4.8.5 and scikit-learn 1.6.1. The project's unit tests pass on CPU, but they use small random models and do not reproduce the experiments.

4.3 Experimental Configuration

Table 4.2 gives the configuration. Both stages use mixed precision. Tokenisation truncates to 128 tokens and pads dynamically. The Task Arithmetic grid covers $\lambda \in \{0.1, 0.2, \ldots, 1.0\}$, and $\lambda = 0.3$ gave the best development score (62.8). Twelve FSFT runs were performed: full-MLP routers with $k \in \{16, 64, 256\}$ and intermediate-only routers with $k = 256$, each with seeds 0, 1 and 2. Seeds vary the router initialisation and the few-shot sample only.

Table 4.2: Training configuration of the two optimisation stages.

Setting	Task-specific experts	FSFT (routers)
Trainable parameters	All of BERT and the head (109.5M per expert)	Routers only (7.13M, shared across tasks)
Optimiser	AdamW [10]; weight decay 0.01, none on 1-D parameters	AdamW; no weight decay
Learning rate	2×10^{-5}	1×10^{-4}
Schedule	Linear decay, 10% warm-up	Linear decay, 10% warm-up
Batch size	32	16 per task (one batch from each task per step)
Duration	3 epochs	500 steps
Gradient clipping	Norm 1.0	None
Model selection	Best epoch by task score on dev	Best step by mean score on dev, every 50 steps

4.4 Evaluation Metrics

Scores follow GLUE conventions and are reported multiplied by 100. From true and false positives and negatives, accuracy, F1 and the Matthews Correlation Coefficient [11] are

$$\mathrm{Acc} = \frac{TP + TN}{TP + TN + FP + FN}, \quad F_1 = \frac{2\,TP}{2\,TP + FP + FN}, \tag{4.1}$$

$$\mathrm{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \tag{4.2}$$

The task score S_t is the mean of accuracy and F1 for MRPC, MCC for CoLA and accuracy for the other tasks. Across tasks, the *average score* of a model m is the mean of $S_t(m)$, and its *normalised score* is the mean of $S_t(m)\,/\,S_t(\text{expert})$. Because a constant prediction already scores about 75 on MRPC and 50 on the accuracy tasks, the code also computes the *gain over majority*:

$$\mathrm{Gain}(m) = \frac{1}{E} \sum_{t=1}^{E} \frac{S_t(m) - S_t(\text{majority})}{S_t(\text{expert}) - S_t(\text{majority})}, \tag{4.3}$$

where 0 is a constant guess and 100% is the expert. Efficiency is measured by stored parameters and size at 32-bit precision.

Chapter 5

Results and Discussion

Scores are from the official validation splits, and WEMoE rows give the mean and sample standard deviation over seeds 0, 1 and 2. Values come from the first full run of the project (Section 5.5).

5.1 Performance Evaluation

Table 5.1 and Figure 5.1 present the main comparison. Task Arithmetic reaches an average of 57.4, moderately above the majority floor of 45.8, and router initialisation alone does not change this (57.8 ± 0.6). Training the routers with FSFT at $k = 256$ raises the full-MLP model to 69.9 ± 0.1, which is 12.5 points above Task Arithmetic and 8.3 below the experts.

Table 5.1: Validation scores ($\times 100$). Task Arithmetic uses $\lambda = 0.3$; FSFT and router-init rows are WEMoE with $k = 256$. Bold marks the best merged model in each column.

Model	MRPC acc	MRPC F1	RTE	CoLA	SST-2	QNLI	Avg	Norm.	Gain
Majority class	68.4	81.2	52.7	0.0	50.9	50.5	45.8	56.4	0.0
Simple Averaging	68.4	81.2	46.9	23.3	55.4	65.9	53.3	66.8	6.6
Task Arithmetic	68.6	81.3	47.3	16.9	69.8	77.9	57.4	70.6	18.4
Router init, full	68.5±0.1	81.3±0.1	47.1±0.2	19.3±2.0	70.2±1.1	77.7±0.2	57.8±0.6	71.3	18.7
FSFT, full MLP	**76.3**±4.5	**83.9**±2.8	52.0±4.4	**42.6**±0.9	**88.0**±0.5	**86.7**±0.7	**69.9**±0.1	**87.8**	**59.1**
FSFT, intermediate	66.5±0.7	76.6±1.0	**59.0**±0.9	41.7±0.9	86.7±0.2	86.5±0.2	69.1±0.1	87.4	54.8
Experts	82.1	88.1	63.2	59.1	92.2	91.2	78.2	100.0	100.0

The normalised score suggests that Task Arithmetic retains 70.6% of expert performance, but the gain reveals that it captures only 18.4% of the distance between a constant guess and the expert, compared with 59.1% for WEMoE + FSFT. Task Arithmetic's accuracy on RTE (47.3) equals 131/277, the share of the *minority* class, which indicates a constant prediction.

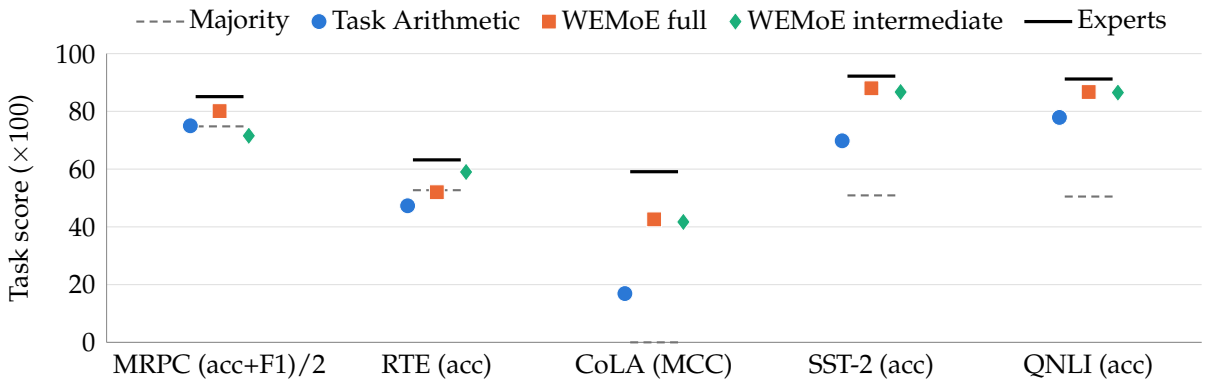


Figure 5.1: Task scores of the merged models between the majority floor (dashed tick) and the experts (solid tick). WEMoE values use $k = 256$ and are means over three seeds; Table 5.1 gives standard deviations.

5.2 Comparison with Individual Experts

Table 5.2 shows each merged model's position per task. On SST-2 and QNLI the trained routers recover close to 90% of the available gain, and on CoLA about 70%. MRPC and RTE, the two smallest tasks, depend on the upscaling scope: the full-MLP model recovers about half of the gain on MRPC but stays marginally below the majority on RTE, whereas the intermediate-only model recovers 60% on RTE but falls below the majority on MRPC. No merged model matches an expert on any task.

Table 5.2: Gain over majority per task, in percent, computed from the seed means. Zero is a constant guess and 100 is the expert.

Model	MRPC	RTE	CoLA	SST-2	QNLI	Mean
Task Arithmetic	2	−52	29	46	67	18.4
WEMoE + FSFT, full MLP, $k = 256$	51	−7	72	90	89	59.1
WEMoE + FSFT, intermediate-only, $k = 256$	−32	60	71	87	88	54.8

The labelled budget strongly influences results (Table 5.3). The average grows with k, but stability is the more informative finding: the standard deviation of CoLA's MCC is 14.1 and 17.8 at $k = 16$ and $k = 64$ (individual runs span 18–44 and 9–41), and 0.9 at $k = 256$. Small budgets also reach their best development score early, consistent with overfitting: 7.1M router parameters face only 80 to 320 training examples.

5.3 Model Storage and Efficiency

Table 5.4 reports stored parameters measured on the built models, which match the closed-form counts in the code. Merged models keep one set of non-MLP parameters but retain the pre-trained MLP weights and one task vector per expert for each upscaled module. Task

Table 5.3: Effect of the labelled budget on the full-MLP model. Best steps are listed per seed; n/a marks those not retained.

k	MRPC acc	RTE	CoLA	SST-2	QNLI	Avg	Norm.	Gain	Best step
16	68.7±2.3	48.0±1.0	27.6±14.1	79.6±8.1	74.0±6.5	60.8±2.7	75.6	25.5	50, n/a, 150
64	71.6±4.4	54.0±3.4	20.5±17.8	89.0±0.6	83.1±2.4	64.6±3.7	79.5	46.7	100, 250, 100
256	76.3±4.5	52.0±4.4	42.6±0.9	88.0±0.5	86.7±0.7	69.9±0.1	87.8	59.1	250, n/a, n/a

vectors therefore dominate, at 283.3M of the 400.0M parameters of the full-MLP model. This limits the saving over five experts to 26.9%, whereas intermediate-only upscaling saves 52.8% and is 35% smaller than full-MLP.

Table 5.4: Stored floating-point parameters (millions) by component, and total size at 32-bit precision.

Deployment	Shared	Base MLP	Task vectors	Routers	Heads	Total	GiB	vs. 5 models
Five single-task models	–	–	–	–	–	547.4	2.04	100.0%
Task Arithmetic	–	–	–	–	–	109.5	0.41	20.0%
WEMoE, full MLP	52.8	56.7	283.3	7.13	0.008	400.0	1.49	73.1%
WEMoE, intermediate-only	81.1	28.3	141.7	7.13	0.008	258.4	0.96	47.2%

Because each input here arrives with its task identifier, the five experts could also be selected directly by task, and they are more accurate than full-MLP WEMoE and only 37% larger. The storage argument therefore rests mainly on the intermediate-only variant, or on scenarios without explicit task identifiers, which this implementation does not evaluate.

5.4 Analysis and Discussion

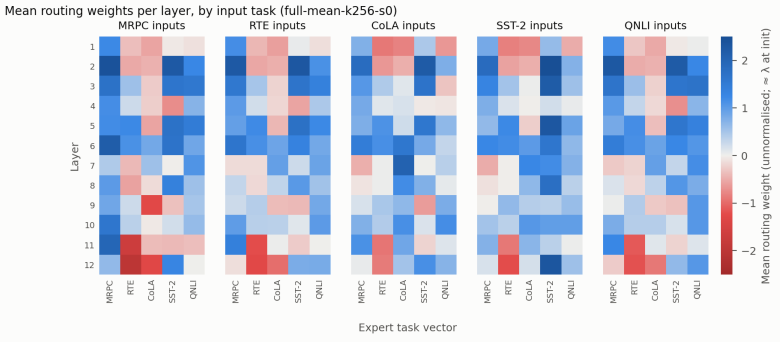


Figure 5.2: Mean gating weights per layer and input task (full MLP, $k = 256$, seed 0). Blue adds and red subtracts a task vector.

Learned routing. Figure 5.2 and the project's run analysis show how far FSFT moves the model from Task Arithmetic. Weights span -3.0 to 4.4, and the per-layer sum reaches 5.8

against 1.5 at initialisation, which a convex mixture could not express. Much of the pattern is shared: mean routing matrices of different input tasks correlate at 0.82 on average, yet the input task still explains about 58% of the routing variance. Routing is thus partly input-dependent, although this does not show that it causes the gain.

Differences across tasks. SST-2 and QNLI, where WEMoE does well, have the largest training sets and the strongest experts, whereas MRPC and RTE have the smallest. The RTE failure of the full-MLP model is visible in routing: on RTE inputs the RTE task vector receives an average weight of about 0.01, compared with 1.37 for the MRPC and SST-2 experts on their own inputs, and it is subtracted in layers 11 and 12. With intermediate-only upscaling this weight rises to 0.96 and RTE accuracy to 59.0. A plausible but untested explanation is that objective (3.7) and mean-score selection trade tasks against one another, retaining router states whose gains elsewhere outweigh a loss on RTE. The weak RTE expert (63.2) may also supply a less useful task vector.

Sampling noise. Approximate standard errors of accuracy are 3.0 points on RTE ($n = 277$) and 2.1 on MRPC ($n = 408$). The 0.8-point gap between scopes and RTE's 52.0 versus the majority's 52.7 are therefore within noise.

Trade-offs. Task Arithmetic is the smallest deployment but performs poorly. Intermediate-only WEMoE stores less than half of five experts with a similar average to the full-MLP model and about half its router training time (8.5 versus 14–16 minutes per run on the T4); its best step came late (400–500 of 500). Full-MLP WEMoE has the highest gain but approaches the size of the experts.

Relation to the original study. Tang et al. report that WEMoE approaches individual fine-tuned models on image tasks, whereas a clear gap remains here, but tasks, backbone, objective and data all differ.

5.5 Limitations

First, "few-shot" is not strict in the sense of Perez et al. [13]: the best step is selected on up to 1,000 labelled examples per task, against $k = 256$ for FSFT. Second, standard deviations exclude expert and data-split variation. Third, per-run outputs were only partly retained: seven of the twelve FSFT runs keep their raw results, and per-example predictions were not retained, so routing statistics come from the project's written run analysis. Results for the input-independent router are unavailable.

Chapter 6

Conclusion and Future Work

This report evaluated whether input-conditioned MLP weights can consolidate five independently fine-tuned BERT experts into one model while preserving task performance. The implementation merges non-MLP parameters by Task Arithmetic, upscales MLP modules into weight-ensembled modules controlled by per-layer routers, and trains only the routers by Few-Shot Fine-Tuning. Predictions come from a single forward pass with parameter-level combination and a task-specific head.

Router training is effective: with 256 labelled examples per task, the average score rises from 57.4 for Task Arithmetic to 69.9 ± 0.1 and the gain over majority from 18.4% to 59.1%, although a clear gap to the experts (78.2) remains. Small budgets ($k = 16, 64$) are unstable across seeds. Averages also conceal failures, since the full-MLP model stays below the majority on RTE and the intermediate-only model falls below it on MRPC. Relative to five experts, WEMoE saves 26.9% of storage with full-MLP upscaling and 52.8% with intermediate-only upscaling. Because the task identifier is known at inference, the case for consolidation under this design depends on the storage constraint justifying the accuracy loss, and is strongest for the intermediate-only variant.

The contributions are an adaptation of WEMoE to BERT with mask-aware routing, a leakage-controlled protocol, and an analysis that uses a majority floor to expose failures that normalised scores conceal. The findings come from one run with one set of experts and should not be generalised beyond this setting.

Future work follows from the limitations. More seeds, including for the experts and the data split, would give more reliable estimates, and a stricter few-shot protocol would cap development data at k. The input-independent router experiment would measure how much of the gain requires input dependence, and measuring inference latency would complete the trade-off evaluation.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.

[2] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.

[3] Ido Dagan, Oren Glickman, and Bernardo Magnini. The PASCAL recognising textual entailment challenge. In *Machine Learning Challenges*, volume 3944 of *Lecture Notes in Computer Science*, pages 177–190. Springer, 2006.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019. arXiv:1810.04805.

[5] William B. Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP)*, 2005.

[6] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). arXiv preprint arXiv:1606.08415, 2016.

[7] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2212.04089.

[8] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. doi: 10.1162/neco.1991.3.1.79.

[9] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, et al. Datasets: A community library for natural language processing. In *Proceedings of EMNLP: System Demonstrations*, 2021. arXiv:2109.02846.

[10] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[11] Brian W. Matthews. Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta (BBA) – Protein Structure*, 405(2):442–451, 1975.

[12] Siyuan Mu and Sen Lin. A comprehensive survey of mixture-of-experts: Algorithms, theory, and applications. arXiv preprint arXiv:2503.07137, 2025.

[13] Ethan Perez, Douwe Kiela, and Kyunghyun Cho. True few-shot learning with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2105.11447.

[14] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of EMNLP*, pages 2383–2392, 2016.

[15] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1701.06538.

[16] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP*, pages 1631–1642, 2013.

[17] Anke Tang, Li Shen, Yong Luo, Nan Yin, Lefei Zhang, and Dacheng Tao. Merging multi-task models via weight-ensembling mixture of experts. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.00433.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762.

[19] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1804.07461.

[20] Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics*, 7:625–641, 2019.

[21] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, 2020. arXiv:1910.03771.